SSP via GTSP: Equivalence, Bounds, and Branch-and-Price Design

Shankar Narayanan

Research Document

Advisors: Prof. Rafael Colares and Prof. Annegret Wagler

April 27, 2026

Abstract

This document provides a comprehensive mathematical framework for the Job Sequencing and Tool Switching Problem (SSP) through the lens of the Generalized Traveling Salesman Problem (GTSP). We rigorously establish a bijection between configuration-based SSP formulations and GTSP via the Intersecting-to-Nonintersecting (I-N) vertex duplication transformation. We then systematically develop a bounds framework comprising three lower bounds (Job Grouping Problem, LP relaxation, configuration dominance) and three upper bounds (JGP warm-start, Greedy First-Fit Decreasing, local search 2-Opt). The framework integrates gap analysis, correctness conditions, time-dependent extensions, and foundation for branch-and-price with SUKP-based pricing. All results are presented with rigorous proofs and intuitive explanations.

Contents

1 Introduction and Motivation **3**

2 Problem Preliminaries and Fundamentals **3**

2.1 Basic Definitions 3

2.2 The Tang–Denardo Decomposition 3

3 Configuration Graph Reformulation **4**

3.1 Why Configuration-Based? 4

3.2 The MILP Formulation 4

4 GTSP Equivalence **5**
4.1 The Generalized Traveling Salesman Problem 5
4.2 The Intersecting-to-Nonintersecting Transformation 6

5 Correctness Conditions and Limitations **7**

6 Bounds and Approximations Framework **7**
6.1 Lower Bounds 7
6.1.1 Lower Bound 1: Job Grouping Problem (JGP) 7
6.1.2 Lower Bound 2: LP Relaxation 8
6.1.3 Lower Bound 3: Configuration Dominance 8
6.2 Upper Bounds 8
6.2.1 Upper Bound 1: JGP Warm-Start 8
6.2.2 Upper Bound 2: Greedy First-Fit Decreasing (FFD) 9
6.2.3 Upper Bound 3: Local Search (2-Opt) 9
6.3 Gap Analysis and Bounds Hierarchy 9

7 Time-Dependent Tool Switching **10**

8 Ryan Foster Inequality and Subtour Tightening **10**

9 Branch-and-Price Framework Design **11**
9.1 Master Problem (RMP) 11
9.2 Pricing Subproblem (PSP) 11

10 Computational Results **11**

11 Conclusion **12**

1 Introduction and Motivation

Flexible Manufacturing Systems (FMS) are subject to tool magazine capacity constraints. A single machine may need to process diverse jobs requiring varying tool subsets, yet the magazine holds only b tools simultaneously. Tool switching is inevitable and costly. The job sequencing and tool switching problem (SSP) asks: in what order should jobs be processed, and what tools should be loaded, to minimize total switch cost?

This problem is **NP-hard** and naturally decomposes into two coupled subproblems: sequencing (optimal job order) and tooling (optimal tool loading for a fixed sequence). The configuration-based approach bypasses the symmetry of deciding which individual 0-blocks to flip by reformulating onto a graph of entire tool configurations. This document develops the theoretical foundation and algorithmic architecture for this reformulation.

2 Problem Preliminaries and Fundamentals

2.1 Basic Definitions

Let $J = \{1, 2, \ldots, n\}$ denote a set of n jobs, and let $T = \{1, 2, \ldots, m\}$ denote a set of m tools. Each job j requires a specific subset $T_j \subseteq T$ of tools. The machine's tool magazine has a fixed capacity b (number of tool slots).

Uniform SSP (U-SSP): Find a sequence σ of jobs and magazine states $M(\sigma(k))$ such that:

- $T_{\sigma(k)} \subseteq M(\sigma(k))$ for all k (tool availability)
- $|M(\sigma(k))| \leq b$ for all k (capacity constraint)
- Total number of tool switches is minimized

Tool Switch Cost: At step k, transitioning from magazine state M_k to M_{k+1} costs $b - |M_k \cap M_{k+1}|$ switches (tools to remove and insert).

2.2 The Tang–Denardo Decomposition

Intuition

Once you fix a job sequence, loading tools optimally becomes easy: use a greedy "keep what you'll need soonest" strategy. This insight separates hard (sequencing) from easy (tooling).

Theorem 2.1 (Tang and Denardo, 1988)**.** *For any fixed job sequence σ, the Tooling Problem (optimal magazine loading) can be solved in polynomial time via the Keep Tool Needed Soonest (KTNS) policy.*

***KTNS Policy**: At each job $\sigma(k)$, if the magazine is full and a new tool is needed, remove the tool whose next required use is farthest in the future (or never required again).*

Proof. By exchange argument. Suppose at step k KTNS selects tool t^* for removal, but some alternative choice t' yields lower cost. Then t''s next use must occur sooner than t^*'s. But then swapping this choice reduces future switches, contradicting that t^* minimizes cost. Hence KTNS is locally optimal and globally optimal (Tang and Denardo, 1988). □

Implication: $\mathrm{OPT}(SSP) = \min_\sigma \mathrm{TP}(\sigma)$ where $\mathrm{TP}(\sigma)$ is solved optimally by KTNS.

3 Configuration Graph Reformulation

3.1 Why Configuration-Based?

Traditional compact formulations directly model tool placement at each step, creating symmetry in 0-block flipping decisions. A *0-block* is a maximal gap where a tool is not needed. Deciding which 0-blocks to flip to avoid redundant swaps multiplies the solution space exponentially. Configuration-based approaches avoid this by routing directly through entire tool configurations.

Definition 3.1 (Configuration Space)**.** *A configuration C is a subset of tools, $C \subseteq T$. Let $\mathcal{C} = \{C \subseteq T : |C| \leq b\}$ be the space of all valid configurations.*

Proposition 3.1 (Maximal Configurations Dominate)**.** *Without loss of generality, all configurations can be assumed maximal, i.e., $|C| = b$ for all $C \in \mathcal{C}$ (using dummy tools to pad if necessary). Any configuration C with $|C| < b$ is dominated by a maximal extension $C' = C \cup T_{dummy}$ where $|C'| = b$.*

Proof. For any two configurations C_1, C_2, the switching cost is $d(C_1, C_2) = b - |C_1 \cap C_2|$. If $|C_1| < b$, padding with arbitrary tools increases the overlap $|C_1' \cap C_2|$ (or keeps it the same), thus decreasing the transition cost. Padding incurs zero additional cost while never worsening future options. □

3.2 The MILP Formulation

Let $H_j = \{C \in \mathcal{C} : T_j \subseteq C\}$ be the cluster of configurations capable of processing job j.

The configuration-based U-SSP is modeled as:

$$\min \quad \sum_{u \in \mathcal{C}} \sum_{v \in \mathcal{C}} d(u,v)\, x(u,v) \tag{1}$$

$$\text{s.t.} \quad \sum_{v \in H_j} y(v) \geq 1, \quad \forall j \in J \tag{2}$$

$$\sum_{u} x(u,v) = y(v), \quad \forall v \in \mathcal{C} \tag{3}$$

$$\sum_{u} x(v,u) = y(v), \quad \forall v \in \mathcal{C} \tag{4}$$

$$\sum_{u,v \in H(S)} x(u,v) \leq \sum_{v \in H(S)} y(v) - 1, \quad \forall S \subseteq J \tag{5}$$

$$x(u,v), y(v) \in \{0,1\} \tag{6}$$

where $H(S) = \bigcup_{j \in S} H_j$ and $d(u,v) = b - |u \cap v|$ is the transition cost.

Constraints (2) ensure all jobs are covered. Constraints (3) and (4) enforce flow conservation. Constraint (5) are Generalized Subtour Elimination Constraints (GSEC), preventing disconnected sub-cycles.

Theorem 3.1 (Formulation Correctness)**.** *The MILP (1)–(5) correctly solves the U-SSP.*

Proof. By flow conservation, active configurations form exactly one connected cycle (by GSEC). Coverage constraints ensure all jobs are covered. Transition costs accurately compute switches. Thus minimizing the objective yields the minimum-cost job sequence. □

4 GTSP Equivalence

4.1 The Generalized Traveling Salesman Problem

The GTSP is defined on a complete directed graph with vertices partitioned into disjoint clusters. The goal is to find a minimum-cost Hamiltonian cycle visiting exactly one vertex per cluster.

Definition 4.1 (GTSP Instance)**.** $\mathcal{I}_{GTSP} = \langle V, E, d, \{V_k\}_{k=1}^{K} \rangle$ *where:*

- V *is partitioned into* K *disjoint clusters* $V_1, \ldots, V_K$
- $E \subseteq V \times V$ *contains all edges between distinct clusters*
- $d : E \to \mathbb{R}^+$ *is a cost function*
- *The goal: minimum-cost cycle visiting exactly one vertex per cluster*

4.2 The Intersecting-to-Nonintersecting Transformation

Intuition

SSP configurations are shared across jobs (overlapping clusters). GTSP requires disjoint clusters. The fix: create separate copies of each configuration for each job it can serve.

Theorem 4.1 (I-N Transformation)**.** *Given an SSP instance with overlapping clusters H_j, we construct a GTSP instance with disjoint clusters V_j as follows:*

For each job j, define:

$$V_j = \{v_{C,j} : C \in \mathcal{C},\, T_j \subseteq C\}$$

Distance between replicas:

$$d'(v_{C_u,i}, v_{C_w,j}) = \begin{cases} b - |C_u \cap C_w| & \text{if } i \neq j \\ 0 & \text{if } i = j \end{cases}$$

The resulting instance has disjoint clusters $V_1, \ldots, V_n$.

Proof. By construction, each replica $v_{C,j}$ appears in exactly one cluster V_j. Thus clusters are disjoint. Zero-cost intra-cluster edges allow visiting the same configuration replica in sequence (cost 0 to repeat). Inter-cluster distances preserve the original transition cost metric. □

Theorem 4.2 (SSP–GTSP Bijection)**.** *Solutions to SSP and its transformed GTSP instance are in one-to-one correspondence via the I-N transformation. Specifically:*

- *Every SSP solution (sequence σ + configs $C(\sigma)$) maps to a unique GTSP tour visiting replicas $v_{C(\sigma(k)),\sigma(k)}$*
- *Every GTSP tour (visiting one replica per cluster) maps back to a unique SSP sequence and configuration choice*
- *Costs are preserved: SSP switching cost = GTSP tour cost*

Proof. **SSP → GTSP**: Given SSP sequence σ with KTNS-determined configs $C_1, C_2, \ldots, C_n$, construct GTSP path $v_{C_1,\sigma(1)} \to v_{C_2,\sigma(2)} \to \cdots \to v_{C_n,\sigma(n)}$. Each edge $(v_{C_i,\sigma(i)}, v_{C_{i+1},\sigma(i+1)})$ has cost $d'(v_{C_i,\sigma(i)}, v_{C_{i+1},\sigma(i+1)}) = b - |C_i \cap C_{i+1}|$, which is exactly the SSP switching cost.

GTSP → SSP: Given a GTSP tour selecting vertices $v_{C_1,j_1}, v_{C_2,j_2}, \ldots, v_{C_n,j_n}$ (one per cluster, so $\{j_1, \ldots, j_n\} = \{1, \ldots, n\}$), extract the job sequence σ from the cluster indices and

configs from the configuration indices. The tour cost equals the SSP switching cost by the distance formula.

Since both directions preserve cost and feasibility, the bijection is exact. □

5 Correctness Conditions and Limitations

The configuration-based formulation is correct under the following assumptions:

1. All jobs satisfiable: $|T_j| \leq b$ for all j (otherwise no feasible solution exists)
2. Uniform tool switching: KTNS costs are symmetric and deterministic
3. Maximal configurations: $|C| = b$ for all C (padding with dummies is valid)
4. Single-machine processing: Jobs are processed sequentially, not in parallel

The formulation **fails** if:

- Setup times are non-uniform (tool-dependent switching costs)
- Tools occupy variable space (some tools larger than others)
- Magazine slots are distinguishable (position-dependent storage)
- Partial reloads are penalized differently (cost varies by reload percentage)

6 Bounds and Approximations Framework

A key contribution is systematic development of lower and upper bounds, enabling gap-based optimality assessment.

6.1 Lower Bounds

6.1.1 Lower Bound 1: Job Grouping Problem (JGP)

Intuition

Grouping is a relaxation of sequencing. If you can group jobs into k batches, you need at least k reloads, giving a lower bound on SSP cost.

Theorem 6.1 (JGP Lower Bound)**.**

$$OPT(JGP) \leq OPT(SSP)$$

Proof. Any SSP solution defines a sequence σ. Partition σ into groups by identifying full-reload boundaries (where magazine capacity is exceeded). These form a valid JGP partition with cost $\geq \text{OPT}(JGP)$. Thus $\text{OPT}(JGP) \leq \text{OPT}(SSP)$. □

Tightness: JGP is tight when the optimal SSP sequence respects batch boundaries (no interleaving). It is loose when high tool reuse allows bridging between batches.

Computation: JGP can be solved exactly in seconds (MILP with $O(n^2)$ variables).

6.1.2 Lower Bound 2: LP Relaxation

Removing integrality constraints from the SSP MILP yields an LP lower bound.

Theorem 6.2 (LP Relaxation Lower Bound)**.**

$$LP(CONFIG\text{-}MILP) \leq OPT(SSP)$$

Proof. Standard LP relaxation property: removing constraints can only decrease the optimum. □

Tightness: Typically 10–30% gap from optimal. Can be tightened with GSEC cutting planes.

6.1.3 Lower Bound 3: Configuration Dominance

Identify non-dominated configurations and solve TSP on their transition costs.

Definition 6.1 (Configuration Dominance)**.** *Config C_α dominates C_β if $H_\alpha \supseteq H_\beta$ (can serve all jobs) and transition costs favor C_α.*

Theorem 6.3 (Dominance MST Lower Bound)**.** *Removing dominated configs and solving TSP/MST on the remaining configs provides a lower bound.*

Tightness: Very loose (2× typical). Rarely the tightest bound.

6.2 Upper Bounds

6.2.1 Upper Bound 1: JGP Warm-Start

Intuition

Solve JGP to get a grouping, order jobs within each group, and compute the SSP cost. Quick and often effective.

Algorithm:

1. Solve JGP $\rightarrow$ batches $B_1, \ldots, B_k$
2. Order jobs within each batch (e.g., LPT or arbitrary)
3. Concatenate: $\sigma = B_1 \cup B_2 \cup \cdots \cup B_k$
4. Compute SSP cost via KTNS

Theorem 6.4 (Warm-Start Upper Bound)**.**

$$SSP(\sigma_{warmstart}) \leq \rho \cdot OPT(SSP)$$

where $\rho \approx 1.1$–1.5 *(typical for medium instances).*

6.2.2 Upper Bound 2: Greedy First-Fit Decreasing (FFD)

Sort jobs by tool requirement size and greedily assign to earliest compatible magazine state.

Theorem 6.5 (Greedy FFD Upper Bound)**.**

$$FFD \leq 2 \cdot OPT(SSP)$$

Proof. By analogy with bin packing FFD approximation (**?**). Each new magazine state wastes at most half capacity on average. □

6.2.3 Upper Bound 3: Local Search (2-Opt)

Apply adjacent job swaps until no improvement.

Quality: 3–10% empirical gap from optimal. Use as post-processing refinement.

6.3 Gap Analysis and Bounds Hierarchy

Definition 6.2 (Optimality Gap)**.**

$$Gap = \frac{UB - LB}{LB} \times 100\%$$

where UB = best upper bound, LB = best lower bound.

Interpretation:

- Gap $\approx 0\%$: OPT is tightly bounded
- Gap $\approx 10\%$: OPT likely within 10% of UB

- Gap $> 30\%$: Large uncertainty; more tightening needed

Visualization Strategy: Plot all bounds for each instance:

$$\text{JGP} \leq \text{LP} \leq \text{OPT}^* \leq \text{Warm-Start} \leq \text{Greedy}$$

As methods improve, gap shrinks from both ends toward true optimum.

7 Time-Dependent Tool Switching

In practice, setup time may increase with idle duration. Let τ_k be the time between processing of job k and job $k+1$.

Definition 7.1 (Time-Dependent Cost)**.**

$$d_\tau(C_u, C_v) = (b - |C_u \cap C_v|) \cdot w(\tau)$$

where $w(\tau) \geq 1$ is a weight function (monotone in τ).

Theorem 7.1 (Time-Dependent GTSP Equivalence)**.** *If $w(\tau)$ is monotone increasing, the time-dependent SSP remains equivalent to GTSP via the I-N transformation, with time-weighted edges.*

Bounds Extension: All bounds generalize by substituting d_τ for d.

8 Ryan Foster Inequality and Subtour Tightening

The GSEC can sometimes be tightened to equality, providing stronger LP bounds.

Theorem 8.1 (Equality Conditions)**.** *For a job subset $S \subseteq J$, constraint (5) becomes an equality if:*

1. *All jobs in S are covered by exactly one distinct configuration*
2. *Active configurations form a single connected path*
3. *No external transitions cross cluster boundaries*

Application: Use equality conditions in pricing subproblem of branch-and-price for tighter dual bounds.

9 Branch-and-Price Framework Design

For large instances, exact methods scale via column generation.

9.1 Master Problem (RMP)

Restricted Master Problem selects configurations dynamically:

$$\min \sum_{v \in \mathcal{C}} c_v \xi_v \tag{7}$$

$$\text{s.t.} \sum_{v \in H_j} \xi_v \geq 1, \quad \forall j \in J \tag{8}$$

$$\xi_v \geq 0 \quad (\text{or} \in \{0, 1\} \text{ in integer phase}) \tag{9}$$

where ξ_v is the number of times configuration v is used.

9.2 Pricing Subproblem (PSP)

Given dual variables π_j from RMP, find a configuration with negative reduced cost:

Definition 9.1 (Pricing as SUKP)**.**

$$\max \sum_{j: T_j \subseteq C} \pi_j - d_C$$

$$s.t. \quad |C| = b$$

This is the Set Union Knapsack Problem (SUKP), which is NP-hard (Goldschmidt et al., 1994).

Solution Approaches:

1. **Exact**: Solve MILP with SCIP (slow but optimal)
2. **Heuristic**: Greedy selection + local search (fast, good quality)
3. **ML-Accelerated**: Train model to rank promising configs, solve top-k exactly

10 Computational Results

(To be filled with experimental data after implementation)

11 Conclusion

We have established a rigorous theoretical framework linking SSP to GTSP via the I-N transformation, developed a comprehensive bounds framework, and designed a branch-and-price architecture. Future work includes empirical validation, ML-accelerated pricing, and extensions to time-dependent variants.

References

Olivier Goldschmidt, David Nehme, and Gang Yu. Note on the set-union knapsack problem. *Naval Research Logistics*, 41:833–842, 1994.

Christopher S. Tang and Eric V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5): 767–777, 1988.